

KahvilaKoodi

Paperinen työpaketti

Viikot 34–49

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Aikataulu

Vko	Pvm	Viikon aihe	Vaihe
34	17.–21.8.	AloitUS: peli-idea ja työkalut	A
35	24.–28.8.	Pelin suunnitelma: ruudut ja feature-lista	A
36	31.8.–4.9.	Feature 1 · Ensimmäinen pelattava kierros	A
37	7.–11.9.	Feature 2 · Kahvilan oikea tuotelista	A
38	14.–18.9.	Feature 3 · Kello, pisteet ja palaute	B
39	21.–25.9.	Feature 4 · Kasvava kiire	B
40	28.9.–2.10.	Feature 5 · Top 5 -tuloslista	B
41	5.–9.10.	Katselmointi · Asiakas pelaa peliäsi	B
42	12.–16.10.	Syysloma – ei projektityötä	–
43	19.–23.10.	Feature 6 · Asiakkaan toivoma parannus	C
44	26.–30.10.	Feature 7 · Peli ohjaa pelaajaa itse	C
45	2.–6.11.	Pelin laatu · Peli kestää pelaamista	C
46	9.–13.11.	Pelin laatu · Koodi, jonka toinen ymmärtää	C
47	16.–20.11.	Pelin julkaisu · Ensimmäinen julkaisuehdokas RC1 testipelattavana	D
48	23.–27.11.	Pelin julkaisu · v1.0 julki	D
49	30.11.–4.12.	Näyttö · Aineisto ja luovutus	D

Vaihe A – Pelin ydin: idea, ensimmäinen pelattava kierros ja

oikea tuotelista

Vko 34 · 17.–21.8. – Aloitus: peli-idea ja työkalut

Viikon jälkeen tiedät, millainen peli tehdään ja kenelle. Tyhjä peli käynnistyy selaimessa.

Tehtävä 1 / 5 · Lue toimeksianto ja kirjoita kahdeksan kysymystä asiakkaalle.

- ☐ Avaa sivun Toimeksianto-näkymä ja lue asiakkaan teksti kerran alusta loppuun.
- ☐ Kirjoita muistiin jokainen asia, jonka pelissä on pakko olla, esimerkiksi aloitusvalikko, pistelasku ja parhaiden tulosten tallennus.
- ☐ Merkitse kohdat, joita teksti ei kerro. Esimerkiksi: kuinka pitkä yksi pelikierros on?
- ☐ Kirjoita kuusi kysymystä asiakkaalle: ota mukaan Toimeksianto-näkymän Sovi asiakkaan kanssa -laatikon neljä kysymystä ja lisää kaksi omaa. Jokaisen vastauksen pitää auttaa sinua päättämään jotain pelistä.
- ☐ Kirjoita jokaisen kysymyksen perään, minkä päätöksen vastaus ratkaisee.
- ☐ Tarkista, ettei kaksi kysymystä kysy samaa asiaa. Vaihda päällekkäinen kysymys uuteen.

Valmis kun: Sinulla on kuusi erilaista kysymystä, ja jokaisen perässä lukee, minkä päätöksen vastaus ratkaisee.

Tallenna työnäyte: Kysymyslista viikon 34 päiväkirjaan, kenttään Mitä tein ja miten?

Tehtävä 2 / 5 · Pidä aloituskeskustelu ja kirjaa asiakkaan vastaukset.

- ☐ Sovi ohjaajan kanssa, kuka on asiakkaana ja milloin keskustelehte.
- ☐ Kysy kysymykset yksi kerrallaan. Kirjoita vastaus heti ylös asiakkaan omin sanoin.
- ☐ Jos asiakas ei osaa vastata, kirjoita kohtaan sana avoin. Älä keksi vastausta itse tai tekoälyllä.
- ☐ Sovi lopuksi, mitkä ominaisuudet ovat pakollisia. Yhdessä ne ovat pakollinen perusversio (P0). Ilman niitä peliä ei voi antaa asiakkaalle.
- ☐ Kirjaa päiväkirjaan keskustelun päivä ja osallistujien roolit, esimerkiksi asiakas ja ohjaaja. Älä kirjoita muiden ihmisten nimiä.

Valmis kun: Päiväkirjassa on jokaiselle kuudelle kysymykselle joko asiakkaan vastaus tai merkintä avoin sekä lista pakollisista (P0) ominaisuuksista.

Tallenna työnäyte: Vastaukset ja avoimet asiat viikon 34 päiväkirjaan.

Tehtävä 3 / 5 · Luo Unity-projekti ja kokeile, aukeaako se selaimessa.

- ☐ Avaa Unity Hub ja valitse Installs. Tarkista, että oppilaitoksen sopima Unity-versio on asennettu ja siinä on Web Build Support.
- ☐ Jos Web Build Support puuttuu, paina version kohdalla hammasratasta, valitse Add modules ja asenna se.
- ☐ Valitse Projects → New project. Valitse pohjaksi Universal 2D, anna nimeksi CafeGame ja paina Create project.
- ☐ Tallenna scene eli pelin näkymä: File → Save As → Assets/Scenes/CafeGame.unity.
- ☐ Avaa File → Build Profiles, valitse Web ja paina Switch Platform.
- ☐ Paina Add Open Scenes ja poista rasti SampleScene-riviltä. Paina Build And Run ja valitse kansioksi Builds/Test. Unity avaa selainversion.
- ☐ Kirjoita muistiin Unity-versio. Se näkyy Unity Hubissa projektin kohdalla, esimerkiksi 6000.0.xx.

Valmis kun: Tyhjä CafeGame-peli aukeaa selaimen, ja tiedät käyttämäsi Unity-version.

Tallenna työnäyte: Unity-versio viikon 34 päiväkirjaan. Kuvakaappaus auenneesta selainversiosta, jonka viet tehtävässä 4 polkuun project-docs/evidence/week-34/web-test.png.

Tehtävä 4 / 5 · Luo Git-repository ja kloonaa se koneellesi.

- ☐ Luo GitHubiin uusi repository. Valitse Public, ellei ohjaaja päättää toisin: ilmaisetillä GitHub Pages toimii vain julkisessa repositoryssa.
- ☐ Valitse kohtaan Add .gitignore pohja Unity. Se pitää Unityn väliaikaiset kansiot, kuten Library ja Temp, poissa versionhallinnasta.
- ☐ Kloonaa eli kopioi repository koneellesi GitHub Desktopilla: File → Clone repository.
- ☐ Sulje Unity. Siirrä CafeGame-projektin kansiot ja tiedostot repositoryn kansioon.
- ☐ Lisää projekti uudesta paikasta Unity Hubiin: Add → Add project from disk. Avaa projekti ja tarkista, että CafeGame-scene aukeaa.

Valmis kun: Repository on GitHubissa, ja Unity Hub avaa CafeGame-projektin repositoryn kansioista.

Tallenna työnäyte: Repositoryn linkki viikon 34 päiväkirjaan.

Tehtävä 5 / 5 · Tee ensimmäinen commit ja push.

- ☐ Luo repositoryn juureen kansio project-docs ja sen sisään tyhjä tiedosto projektipaivakirja.md. Git ei tallenna tyhjää kansiota.
- ☐ Siirrä tehtävän 3 kuvakaappaus polkuun project-docs/evidence/week-34/web-test.png.
- ☐ Kirjoita README.md-tiedostoon pelin nimi ja yksi virke siitä, mitä peli tekee. README on repositoryn esittelytiedosto.
- ☐ Tarkista GitHub Desktopin Changes-listasta, että Assets, Packages, ProjectSettings, project-docs ja README.md ovat mukana. Library, Temp ja Builds eivät saa olla mukana.
- ☐ Tee commit eli tallenna muutokset versionhallintaan: kirjoita GitHub Desktopin Summary-kenttään "Unity-projektin pohja" ja paina Commit to main. Tee sitten push painamalla Push origin, jolloin commit siirtyy GitHubiin.

Valmis kun: Commit näkyy GitHubissa. Repositoryssa ovat Assets-, Packages-, ProjectSettings- ja project-docs-kansiot sekä README.md.

Tallenna työnäyte: Repositoryn linkki ja ensimmäisen commitin tunniste (7 merkin koodi GitHubin commit-listassa, esimerkiksi a1b2c3d) viikon 34 päiväkirjaan.

Valmis kun: Asiakkaan vastaukset ja avoimet asiat ovat päiväkirjassa, tyhjä peli aukeaa selaimessa ja ensimmäinen commit näkyy GitHubissa.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: kysymykset ja asiakkaan vastaukset päiväkirjassa, kuvakaappaus selainversiosta, repository-linkki ja ensimmäinen commit.

Vko 35 · 24.–28.8. – Pelin suunnitelma: ruudut ja feature-lista

Viikon jälkeen peli on paperilla: kolme ruutua, skriptien kartta ja tehtävälusta GitHubissa. Ohjaaja on hyväksynyt rajauksen.

Tehtävä 1 / 5 · Täytä pelin suunnitteludokumentti (GDD) ja vie se repositoryyn.

- ☐ Avaa Suunnitelma (GDD) -näkyä. GDD eli Game Design Document on pelin suunnitteludokumentti. Lue esitetyt osa: se tulee toimeksiannosta.
- ☐ Kirjoita pelille työnimi ja oma nimesi.
- ☐ Kirjoita kahdella tai kolmella virkkeellä, mitä peli tavoittelee, kenelle se on ja mikä on sinun roolisi.
- ☐ Valitse visuaalinen tyyli. Kirjoita, mistä grafiikka tulee ja millä lisenssillä eli käyttöluvalla sitä saa käyttää, esimerkiksi Kenney.nl, CC0 (saa käyttää vapaasti).
- ☐ Pääta pistet: montako pistettä pelaaja saa oikeasta toimituksesta ja montako hän menettää väärästä. Perustele valinta yhdellä tai kahdella virkkeellä.
- ☐ Jos sovit kierroksen pituuden asiakkaan kanssa viikolla 34, kirjaa se. Muuten jätä kenttä tyhjäksi: se on avoin asia.
- ☐ Paina Lataa gdd.md. Siirrä tiedosto repositoryn project-docs-kansioon, tee commit ja push.

Valmis kun: Tiedosto `project-docs/gdd.md` näkyy GitHubissa, eikä yhdessäkään OMA-merkityssä kentässä ole tyhjää kohtaa. OHJAAJA-merkitty kierroksen pituus saa jäädä avoimeksi.

Tallenna työnäyte: `project-docs/gdd.md` ja commit-linkki viikon 35 päiväkirjaan.

Tehtävä 2 / 5 · Pilko ensimmäiset featuret pieniksi tehtäviksi GitHubiin.

- ☐ Avaa Suunnitelma-näkymän lista Featuret tekojärjestyksessä. Feature on yksi pelaajalle näkyvä ominaisuus, esimerkiksi pistelasku.
- ☐ Ota listan kaksi ensimmäistä featurea. Kirjoita jokaisesta paperille 3–5 pientä asiaa, joita sen tekemiseen tarvitaan, esimerkiksi pelattava kierros: kolme ruutua, ruudun vaihto, tilaus ja pisteet.
- ☐ Avaa GitHubissa repository → Issues → New issue. Tee jokaisesta pienestä asiasta oma issue eli tehtävä. Aloita otsikko verbillä, esimerkiksi Näytä asiakkaan tilaus.
- ☐ Kirjoita jokaiseen issueen arvio: puoli päivää tai yksi päivä. Jos tehtävä on isompi, jaa se kahdeksi issueksi.
- ☐ Kirjoita jokaiseen issueen Valmis kun -ehto: mitä toinen ihminen näkee pelissä, kun tehtävä on tehty.
- ☐ Lisää vielä yksi issue kustakin muusta featuresta paitsi asiakkaan toivomasta parannuksesta, joka selviää vasta viikolla 41. Pilkot ne tarkemmin sillä viikolla, kun teet featuren.

Valmis kun: GitHubissa on issues kahdelle ensimmäiselle featurelle ja yksi issue kustakin muusta. Jokaisessa on arvio ja Valmis kun -ehto.

Tallenna työnäyte: Linkki GitHubin Issues-listaan viikon 35 päiväkirjaan.

Tehtävä 3 / 5 · Merkitse jokaiselle tehtävälle, onko se pakollinen, tärkeä vai lisä.

- ☐ Luo GitHubissa kolme labelia eli tunnustetta: Issues → Labels → New label. Anna nimiksi P0 pakollinen, P1 tärkeä ja P2 lisä.
- ☐ Anna label P0 pakollinen jokaiselle issuelle, jota ilman peliä ei voi antaa asiakkaalle. Kaikki toimeksiannon vaatimukset ovat pakollisia (P0). Käytä apuna viikon 34 aloituskeskustelun pakollisten (P0) listaa.
- ☐ Anna label P1 tärkeä issueille, jotka parantavat peliä. Peli on kuitenkin valmis ilman niitä. Tärkeät (P1) tehdään vasta, kun pakolliset (P0) toimivat.
- ☐ Anna label P2 lisä issueille, jotka voi jättää pois, jos aika loppuu. Lisät (P2) tehdään viimeisenä.
- ☐ Näytä lista ohjaajalle ja pyydä hyväksyntä rajaukselle eli sille, mitä peliin tehdään ja mitä jätetään pois. Kirjaa päiväkirjaan hyväksyjän rooli ja päivä.

Valmis kun: Jokaisella issuella on yksi label: P0 pakollinen, P1 tärkeä tai P2 lisä. Ohjaaja on hyväksynyt listan.

Tallenna työnäyte: Linkki Issues-listaan sekä hyväksyjän rooli ja päivä viikon 35 päiväkirjaan.

Tehtävä 4 / 5 · Piirrä pelin kolme ruutua paperille.

- ☐ Ota paperi ja jaa se kolmeen osaan. Kirjoita osien otsikoiksi Valikko, Peli ja Tulos.
- ☐ Piirrä Valikko-ruutuun pelin nimi ja Aloita-painike.
- ☐ Piirrä Peli-ruutuun asiakkaan tilaus, tuotepainikkeet (esimerkiksi Kahvi, Tee ja Sämpylä), Toimita-painike, aika ja pisteet.
- ☐ Piirrä Peli-ruutuun myös kohta, jossa peli kertoo, menikö toimitus oikein.
- ☐ Piirrä Tulos-ruutuun loppupisteet, viiden parhaan tuloksen lista ja Pelaa uudelleen -painike.

- Piirrä nuoli jokaisesta ruutua vaihtavasta painikkeesta siihen ruutuun, johon se vie, ja Peli-ruudusta Tulos-ruutuun, kun aika loppuu. Kuvaa paperi ja tallenna kuva nimellä mockup.png eli käyttöliittymän luonnoskuva.

Valmis kun: Kuvassa on kolme ruutua, ja nuolet näyttävät, mihin Aloita ja Pelaa uudelleen vievät ja milloin peli siirtyy tulosruutuun.

Tallenna työnäyte: Kuva polkuun project-docs/evidence/week-35/mockup.png, commit ja push.

Tehtävä 5 / 5 · Piirrä kartta pelin C#-skripteistä ja niiden tehtävistä.

- Piirrä paperille viisi laatikkoa: GameManager, OrderManager, ProductDatabase, DifficultyController ja SaveService. Jokainen laatikko on yksi C#-skripti eli ohjelmatiedosto.
- Kirjoita jokaisen laatikon alle omin sanoin, mitä skripti tekee kahvilapelissä. Apuna on lista kortin lopun suljetussa kohdassa Skriptien tehtävät.
- Piirrä nuolet: products.json → ProductDatabase, ProductDatabase → OrderManager (tuotelista) ja SaveService → selaimen tallennus. Tuotteet tulevat tiedostosta, ja tulokset säilyvät selaimessa.
- Kuvaa paperi ja tallenna kuva nimellä unity-rakenne.png. Näytä kuva ohjaajalle ennen koodaamista ja kirjaa päiväkirjaan hänen kommenttinsa, roolinsa ja päivä.

Valmis kun: Kuvassa on viisi skriptiä, jokaisella oma tehtävä, ja nuolet näyttävät, mistä tuotteet ja tulokset kulkevat.

Tallenna työnäyte: Kuva polkuun project-docs/evidence/week-35/unity-rakenne.png, commit ja push.

Valmis kun: Tiedosto gdd.md on repositoryssa, jokaisella issuella on arvio, tärkeysluokka – pakollinen (P0), tärkeä (P1) tai lisää (P2) – ja Valmis kun -ehto, ja ohjaaja on hyväksynyt rajauksen. Luonnoksessa näkyvät valikko, peli ja tulos.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: gdd.md, Issues-lista prioriteetteineen, rajauksen hyväksyntä, mockup.png ja unity-rakenne.png.

Vko 36 · 31.8.–4.9. – Feature 1 · Ensimmäinen pelattava kierros

Peliä voi pelata ensimmäistä kertaa: Aloita → asiakas tilaa kahvin → toimitat → saat pisteet → tulosruutu.

Tehtävä 1 / 5 · Rakenna Unityyn valikkoruutu.

- ☐ Avaa CafeGame-scene. Valitse Hierarchy-ikkunassa eli scenen objektistassa + → UI → Canvas. Canvas on alue, jolle pelin tekstit ja painikkeet tulevat.
- ☐ Klikkaa Canvasia hiiren oikealla ja valitse UI → Panel. Tee näin kolme paneelia ja nimeä ne MenuPanel, GamePanel ja ResultPanel.
- ☐ Lisää MenuPaneliin painike: UI → Button - TextMeshPro. Paina Import TMP Essentials, jos Unity kysyy. Nimeä painike StartButton.
- ☐ Valitse StartButtonin alta Text (TMP) ja kirjoita Inspectorin tekstikenttään Aloita.
- ☐ Lisää MenuPaneliin teksti (UI → Text - TextMeshPro) ja kirjoita siihen pelin nimi.
- ☐ Siirrä Scene-ikkunassa pelin nimi ja Aloita-painike luonnoksesi (mockup.png) mukaisille paikoille.

Valmis kun: Hierarchyssä on Canvas ja sen alla kolme paneelia. MenuPanelissa näkyvät pelin nimi ja Aloita-painike.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 36 päiväkirjaan.

Tehtävä 2 / 5 · Rakenna peli- ja tulosruutu.

- ☐ Lisää GamePaneliin kolme tekstiä (UI → Text - TextMeshPro): OrderText, ScoreText ja TimeText.
- ☐ Lisää GamePaneliin kaksi painiketta: CoffeeButton tekstillä Kahvi ja SubmitButton tekstillä Toimita.
- ☐ Lisää ResultPaneliin teksti FinalScoreText ja painike RestartButton tekstillä Pelaa uudelleen.
- ☐ Siirrä Scene-ikkunassa jokaisen paneelin tekstit ja painikkeet erilleen luonnoksesi (mockup.png) mukaan, jotta ne eivät ole päällekkäin.
- ☐ Piilota GamePanel ja ResultPanel: valitse paneeli ja poista rasti Inspectorin yläreunasta nimen vierestä.

Valmis kun: Hierarchy näyttää samalta kuin avun rakenne (Main Camera, Global Light 2D ja EventSystem saavat olla lisäksi), ja Play-tilassa näkyy vain valikko.

Tallenna työnäyte: Kuvakaappaus Hierarchy-ikkunasta polkuun project-docs/evidence/week-36/hierarchy.png, commit ja push. Polku viikon 36 päiväkirjan kenttään Missä työnäyte on?

Tehtävä 3 / 5 · Tee GameManager-skripti, joka vaihtaa ruutua.

- ☐ Luo Project-ikkunan Assets-kansioon kansio Scripts: hiiren oikea → Create → Folder.
- ☐ Klikkaa Scripts-kansiota hiiren oikealla ja valitse Create → Scripting → MonoBehaviour Script eli objektiin liitettävä C#-skripti. Anna nimeksi GameManager.
- ☐ Avaa skripti ja korvaa sen sisältö avun työpohjalla. Tallenna tiedosto.
- ☐ Luo Hierarchyyn tyhjä objekti: + → Create Empty. Nimeä se GameManager ja raahaa GameManager-skripti sen päälle.
- ☐ Valitse GameManager-objekti. Raahaa MenuPanel, GamePanel ja ResultPanel Hierarchyistä Inspectorin kenttiin Menu Panel, Game Panel ja Result Panel.
- ☐ Kytke Aloita-painike. Valitse StartButton ja paina On Click () -listan +. Raahaa GameManager-objekti kenttään ja valitse GameManager → StartGame.
- ☐ Kytke RestartButton samalla tavalla metodiin eli skriptin toimintoon RestartGame.
- ☐ Paina Play ja sitten Aloita. Peliruutu tulee näkyviin, ja valikko katoaa.

Valmis kun: Play-tilassa Aloita vaihtaa valikon peliruutuun.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 36 päiväkirjaan.

Tehtävä 4 / 5 · Tee ensimmäinen tilaus: asiakas tilaa kahvin ja pelaaja toimittaa sen.

- ☐ Valitse GameManager-objekti. Raahaa Inspectorissa OrderText, ScoreText ja FinalScoreText skriptin kenttiin.
- ☐ Täydennä StartGame-metodin TODO-rivit eli koodiin merkityt täydennettävät kohdat: kirjoita OrderText-tekstiin "Asiakas tilaa: Kahvi" ja näytä pisteet ScoreText-tekstissä.
- ☐ Kytke CoffeeButtonin On Click () -listaan GameManager → SelectCoffee. Kytke SubmitButtonin listaan GameManager → SubmitOrder.
- ☐ Täydennä SubmitOrder: jos kahvi on valittu, lisää pisteisiin 10 ja päivitä ScoreText.
- ☐ Täydennä EndGame: näytä pisteet FinalScoreText-kentässä.
- ☐ Paina Play ja pelaa: Aloita → Kahvi → Toimita.

Valmis kun: Aloita → Kahvi → Toimita → tulosruudulla lukee 10 pistettä.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 36 päiväkirjaan.

Tehtävä 5 / 5 · Testaa kierros kolme kertaa selaimessa ja kirjaa tulokset.

- ☐ Tee selainversio: File → Build Profiles → Web → Build And Run.
- ☐ Kirjoita päiväkirjaan ennen pelaamista odotettu tulos: Aloita → Kahvi → Toimita → tulosruudulla 10.
- ☐ Pelaa kierros kolme kertaa. Käytä välillä Pelaa uudelleen -painiketta.
- ☐ Kirjaa jokaisesta kierroksesta, menikö se odotetusti. Jos ei mennyt, kirjoita tarkasti, mitä tapahtui.
- ☐ Korjaa löytämäsi virheet vasta kirjaamisen jälkeen. Kirjaa korjaus samaan kohtaan.
- ☐ Tee commit ja push. Kirjoita commit-viestiin, mitä korjasit.
- ☐ Sulje GitHubissa tämän viikon valmiit issuet. Kirjoita jokaiseen ennen sulkemista kommentti, jossa on linkki commitiin, jossa työ tehtiin.

Valmis kun: Päiväkirjassa on kolme testikierrosta tuloksineen, ja viimeinen kierros meni odotetusti.

Tallenna työnäyte: Kolme kuvaa tai lyhyt video pelipolusta kansioon project-docs/evidence/week-36/, commit ja push. Testikirjaukset ja commit-linkki viikon 36 päiväkirjaan.

Valmis kun: Aloita → Kahvi → Toimita → tulosruutu näyttää pisteet. Polku toimii selainversiossa ilman, että kosket Unity-editoriin kesken pelin.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: video tai kuvat pelipolusta, viisi testikierrosta päiväkirjassa ja commit-linkit.

Vko 37 · 7.–11.9. – Feature 2 · Kahvilan oikea tuotelista

Asiakas tilaa 1–3 tuotetta, jotka arvotaan tuotelistasta. Tuotteita voi muuttaa koskematta koodiin.

Tehtävä 1 / 5 · Kirjoita kahvilan tuotteet products.json-tiedostoon.

- ☐ Luo Assets-kansioon kansio Data. Luo sinne koodieditorilla, esimerkiksi VS Codella, tiedosto products.json.
- ☐ Kirjoita tiedostoon kolme tuotetta avun mallin mukaan. Jokaisella tuotteella on id (tunniste), name (nimi) ja points (pisteet).
- ☐ Anna points-arvoksi se pistemäärä, jonka päätit pelin suunnitteludokumentissa (GDD) oikealle toimitukselle, esimerkiksi 10.
- ☐ Tarkista tiedoston muoto: lainausmerkit nimien ympärillä, pilkut tuotteiden välissä ja hakasulkeet listan ympärillä. VS Code näyttää muotovirheen punaisella.
- ☐ Palaa Unityyn ja klikkaa products.json-tiedostoa Project-ikkunassa. Inspector näyttää tiedoston sisällön.

Valmis kun: Unityn Inspector näyttää products.json-tiedoston, ja siinä on kolme tuotetta.

Tallenna työnäyte: Commit ja push. Tiedoston polku viikon 37 päiväkirjaan.

Tehtävä 2 / 5 · Lue tuotteet tiedostosta ProductDatabase-skriptillä.

- ☐ Luo Scripts-kansioon skripti ProductDatabase ja korvaa sen sisältö avun työpohjalla.
- ☐ Lue työpohjasta kaksi luokkaa: ProductData on yksi tuote ja ProductList on tuotteiden lista. Rivi [System.Serializable] kertoo Unitylle, että luokan voi lukea JSONista.
- ☐ Luo Hierarchyyn tyhjä objekti ProductDatabase ja raahaa skripti sen päälle.
- ☐ Raahaa products.json Inspectorissa ProductDatabase-skriptin Products Json -kenttään.
- ☐ Täydennä ensimmäinen TODO-kohta: jos Products Json -kenttä on tyhjä, kirjoita virheilmoitus Consoleen eli Unityn viesti-ikkunaan ja palauta tyhjä lista.
- ☐ Täydennä toinen TODO-kohta: ota rikkinäisen JSONin virhe kiinni try–catch-lohkolla eli koodilla, joka nappaa virheen ennen kuin peli kaatuu. Kirjoita virheilmoitus Consoleen ja palauta tyhjä lista.

Valmis kun: Avun tarkistustestissä Console näyttää Kahvi, ja molemmat TODO-kohdat on täydennetty.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 37 päiväkirjaan.

Tehtävä 3 / 5 · Arvo asiakkaalle 1–3 tuotteen tilaus OrderManager-skriptillä.

- ☐ Luo skripti OrderManager avun työpohjasta ja liitä se GameManager-objektiin.
- ☐ Raahaa ProductDatabase-objekti OrderManager-skriptin Product Database -kenttään.
- ☐ Täydennä CreateOrder: arvo tilaukseen 1–3 tuotetta listasta ja palauta tilausteksti, esimerkiksi ”Asiakas tilaa: Kahvi, Sämpylä”.
- ☐ Lisää GameManageriin kenttä [SerializeField] private OrderManager orderManager; ja raahaa GameManager-objekti Inspectorissa sen Order Manager -kenttään.
- ☐ Muuta StartGame-metodia: kirjoita OrderText-tekstiin CreateOrder-metodin palauttama tilausteksti.
- ☐ Paina Play ja Aloita viisi kertaa. Kirjaa, mitä tilauksia tuli.

Valmis kun: Play-tilassa tilaus vaihtuu joka pelissä, ja siinä on 1–3 tuotetta.

Tallenna työnäyte: Commit ja push. Commit-linkki ja viiden tilauksen lista viikon 37 päiväkirjaan.

Tehtävä 4 / 5 · Tee tuotepainikkeet ja tarkista, menikö toimitus oikein.

- ☐ Lisää GamePaneliin painikkeet TeaButton tekstillä Tee ja BreadButton tekstillä Sämpylä.
- ☐ Poista Kahvi-painikkeen vanha kytkeä: valitse CoffeeButton, valitse On Click () -listan SelectCoffee-rivi ja paina –.
- ☐ Kytke kolme tuotepainiketta GameManager-objektin OrderManager → SelectProduct -metodiin. Kirjoita parametriksi eli annettavaksi arvoksi tuotteen id: kahvi, tee tai sampyla.
- ☐ Täydennä CheckDelivery: vertaa valittuja tuotteita tilaukseen. Oikeasta toimituksesta metodi palauttaa tuotteiden points-arvojen summan. Väärästä toimituksesta se palauttaa nollan.
- ☐ Muuta SubmitOrder-metodia: lisää pisteisiin CheckDelivery-metodin palauttama arvo.
- ☐ Paina Play ja toimita yksi oikea ja yksi väärä tilaus. Kirjaa, montako pistettä kumpikin antoi.

Valmis kun: Oikea toimitus antaa tuotteiden pisteet yhteensä, ja väärä toimitus antaa 0 pistettä.

Tallenna työnäyte: Commit ja push. Commit-linkki ja kahden toimituksen tulos viikon 37 päiväkirjaan.

Tehtävä 5 / 5 · Testaa tuotelista kolmella testillä.

- ☐ Kirjoita päiväkirjaan odotettu tulos kolmelle testille ennen kuin testaat.

- Testi 1: muuta kahvin points-arvo 10 → 15 ja paina Play. Toimita oikein tilaus, jossa on kahvi. Pisteet ovat tuotteiden summa, jossa kahvi on 15, esimerkiksi Kahvi + Tee = 25.
- Testi 2: tyhjennä ProductDatabase-skriptin Products Json -kenttä ja paina Play. Consolessa näkyy virheilmoitus, eikä peli kaadu.
- Testi 3: poista products.json-tiedostosta yksi pilkku ja paina Play. Consolessa näkyy virheilmoitus, eikä peli kaadu.
- Palauta tiedosto ja kenttä ennalleen. Kirjaa jokaisen testin todellinen tulos päiväkirjaan.
- Kirjoita päiväkirjaan omin sanoin, mitä kukin vaihe tekee tiedon kulussa: products.json → ProductDatabase → OrderManager → GameManager → OrderText.
- Sulje GitHubissa tämän viikon valmiit issue't. Kirjoita jokaiseen ennen sulkemista kommentti, jossa on linkki commitiin, jossa työ tehtiin.

Valmis kun: Kaikki kolme testiä menivät odotetusti, ja tulokset ovat päiväkirjassa.

Tallenna työnäyte: Testien tulokset ja tiedon kulku viikon 37 päiväkirjaan.

Valmis kun: Kun muutat kahvin pistearvon products.json-tiedostossa, muutos näkyy pelissä ilman C#-muutosta. Jos tiedosto puuttuu tai on rikki, peli näyttää virheilmoituksen eikä kaadu.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: products.json, ProductDatabase.cs ja OrderManager.cs committeina sekä kolmen testin tulokset.

Vaihe B – Pelin featuret: kiire, kasvava vaikeus, top 5 ja

asiakaskatselmointi

Vko 38 · 14.–18.9. – Feature 3 · Kello, pisteet ja palaute

Peliin tulevat kello, pisteet ja välitön palaute. Kierros päättyy, kun aika loppuu.

Tehtävä 1 / 4 · Kirjoita pelin säännöt ennen koodaamista.

- ☐ Avaa project-docs/gdd.md. Katso, mitkä pisteet päätit oikealle ja väärälle toimitukselle.
- ☐ Kirjoita päiväkirjaan sääntö 1: oikea toimitus antaa tuotteiden pisteet yhteensä.
- ☐ Kirjoita sääntö 2: väärä toimitus vie pois sen määrän pisteitä, jonka päätit pelin suunnitteludokumentissa (GDD).
- ☐ Kirjoita sääntö 3: kierros kestää __ sekuntia. Käytä asiakkaan kanssa sovittua pituutta. Jos sitä ei ole sovittu, käytä 60 sekuntia ja merkitse asia avoimeksi.
- ☐ Kirjoita sääntö 4: peli päättyy kerran, kun aika on 0. Kirjoita sääntö 5: voivatko pisteet mennä alle nollan, ja perustele oma päätöksesi yhdellä virkkeellä.

Valmis kun: Päiväkirjassa on viisi numeroitua sääntöä lukuarvoineen.

Tallenna työnäyte: Säännöt viikon 38 päiväkirjaan.

Tehtävä 2 / 4 · Lisää peliin ajastin, joka päättää kierroksen.

- ☐ Lisää GameManageriin avun työpohjan neljä kenttää: timeText, roundSeconds (kierroksen pituus), timeLeft (jäljellä oleva aika) ja isPlaying (onko peli käynnissä).
- ☐ Raahaa TimeText Inspectorissa GameManager-skriptin Time Text -kenttään ja kirjoita Round Seconds -kenttään säännön 3 kierroksen pituus.
- ☐ Täydennä StartGame: timeLeft saa kierroksen pituuden ja isPlaying arvon true.
- ☐ Lisää avun Update-metodi GameManageriin. Se vähentää aikaa Time.deltaTime-arvolla eli edellisestä ruudunpäivityksestä kuluneella ajalla. Täydennä sen TODO: näytä aika TimeText-tekstissä kokonaisina sekunteina.
- ☐ Kirjoita EndGame-metodin ensimmäiseksi riviksi isPlaying = false, jotta Update ei kutsu EndGamea uudelleen eikä peli pääty kahdesti.
- ☐ Poista SubmitOrder-metodista viikolla 36 lisätty EndGame-kutsu. Nyt kierros päättyy vasta, kun aika loppuu.

Valmis kun: Play-tilassa aika laskee, pelaaja voi toimittaa monta tilausta ja tulosruutu aukeaa kerran, kun aika on 0.

Tallenna työnäyte: Commit ja push. Commit-linkki viikon 38 päiväkirjaan.

Tehtävä 3 / 4 · Näytä pelaajalle heti, menikö toimitus oikein.

- ☐ Lisää GamePaneliin teksti FeedbackText: UI → Text - TextMeshPro.
- ☐ Lisää GameManageriin kenttä [SerializeField] private TMP_Text feedbackText; ja raahaa FeedbackText Inspectorissa Feedback Text -kenttään.
- ☐ Toimitus on oikein, kun CheckDelivery palauttaa enemmän kuin 0. Silloin lisää pisteet ja kirjoita FeedbackText-tekstiin esimerkiksi "Oikein! +10".
- ☐ Kun toimitus on väärin, vähennä pisteitä säännön 2 mukaan, noudata sääntöä 5 (alle nollan vai ei) ja kirjoita esimerkiksi "Väärä tilaus -5".
- ☐ Päivitä ScoreText heti jokaisen toimituksen jälkeen.
- ☐ Tee toimituksen jälkeen uusi tilaus CreateOrder-metodilla, jotta peli jatkuu, kunnes aika loppuu.

Valmis kun: Oikea ja väärä toimitus näyttävät eri palautteen, ja pisteet muuttuvat sääntöjen mukaan.

Tallenna työnäyte: Kuvakaappaukset molemmista palautteista kansioon project-docs/evidence/week-38/, commit ja push. Polut viikon 38 päiväkirjaan.

Tehtävä 4 / 4 · Testaa pelin kolme rajatilannetta.

- ☐ Kirjoita päiväkirjaan jokaiselle kolmelle testille odotettu tulos ennen kuin testaat.
- ☐ Testi 1: aseta Inspectorin Round Seconds -arvoksi 3, paina Play ja Aloita ja odota. Tulosruutu aukeaa vain kerran.
- ☐ Testi 2: paina Toimita kaksi kertaa nopeasti peräkkäin. Vain ensimmäinen painallus lasketaan.
- ☐ Testi 3: paina tulosruudulla Pelaa uudelleen. Aika ja pisteet alkavat alusta.
- ☐ Jos testi ei mennyt odotetusti, korjaa vika, tee commit ja testaa uudelleen. Kirjaa molemmat ajot.
- ☐ Sulje GitHubissa tämän viikon valmiit issueet. Kirjoita jokaiseen ennen sulkemista kommentti, jossa on linkki commitiin, jossa työ tehtiin.

Valmis kun: Kolmen rajatestin odotetut ja todelliset tulokset on kirjattu, ja viimeinen ajo meni odotetusti.

Tallenna työnäyte: Testien tulokset ja korjausten commit-linkit viikon 38 päiväkirjaan.

Valmis kun: Peli päättyy kerran, kun aika loppuu. Nopea kaksoispainallus ei anna kahta tulosta, ja uusi peli nollaa ajan sekä pisteet.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: pelisäännöt, ajastimen ja palautteen commitit sekä neljän rajatestin tulokset.

Vko 39 · 21.–25.9. – Feature 4 · Kasvava kiire

Peli vaikeutuu pistemäärän mukaan: isommat tilaukset tai lyhyemmät ajat, ei äkkinäisiä hyppyjä.

Tehtävä 1 / 3 · Vertaa kahta tapaa vaikeuttaa peliä ja tee valinta.

- ☐ Lue kaksi tapaa vaikeuttaa peliä: A = tilauksissa on enemmän tuotteita, B = asiakkaalla on vähemmän aikaa.
- ☐ Pyydä vertainen eli toinen opiskelija tai ohjaaja keskustelemaan kanssasi kymmeneksi minuutiksi.
- ☐ Arvioi kumpikin tapa kolmella kysymyksellä: Huomaako pelaaja vaikeutumisen? Onko arvoja helppo säätää? Onko sitä helppo testata?
- ☐ Valitse A, B tai molemmat. Kirjoita päiväkirjaan valinta ja kaksi perustelua.
- ☐ Kirjoita päiväkirjaan keskustelukumppanin rooli, esimerkiksi vertainen tai ohjaaja.

Valmis kun: Päiväkirjassa ovat molemmat tavat, kolmen kysymyksen vastaukset, valinta ja kaksi perustelua.

Tallenna työnäyte: Vertailu ja valinta viikon 39 päiväkirjaan.

Tehtävä 2 / 3 · Tee kolme vaikeustasoa, joita voi säätää Inspectorissa.

- ☐ Luo skripti DifficultyController avun työpohjasta ja liitä se GameManager-objektiin.
- ☐ Lisää GameManageriin kenttä [SerializeField] private DifficultyController difficultyController; ja raahaa GameManager-objekti Inspectorissa sen Difficulty Controller -kenttään.
- ☐ Aseta Inspectorissa Levels-listan kooksi 3. Täytä tasot avun taulukon mukaan tai omien arvojesi mukaan.
- ☐ Täydennä GetLevel: palauta korkein taso, jonka aloituspisteet pelaaja on jo saavuttanut. Aloituspisteet ovat tason minScore-arvo.
- ☐ Jos valitsit tavan A: muuta CreateOrder muotoon CreateOrder(int count) ja poista sen Random.Range-arvonta. GameManager antaa sille arvon difficultyController.GetLevel(score).itemCount.
- ☐ Jos valitsit tavan B: aseta jokaiselle asiakkaalle odotusaika tason customerTime-arvosta. Kun aika loppuu, asiakas lähtee ja tulee uusi tilaus.
- ☐ Muuta yhden tason arvoa Inspectorissa ja paina Play. Muutos näkyy pelissä ilman koodimuutosta.

Valmis kun: Pisteillä 0, 31 ja 61 peli käyttää eri tasoa, ja tason arvot voi muuttaa Inspectorissa.

Tallenna työnäyte: Commit ja push. Commit-linkki ja tasotaulukko viikon 39 päiväkirjaan.

Tehtävä 3 / 3 · Testaa, että pelin kolme tilaa pysyvät erillään.

- ☐ Kirjoita päiväkirjaan odotettu tulos ennen kuin testaat.
- ☐ Pelaa kolme kierrosta niin, että pääset eri kierroksilla tasoille 0, 1 ja 2 eli Levels-listan kolmelle riville minScore-arvojesi mukaan.
- ☐ Tarkista joka kierroksella: kun odotat valikossa 10 sekuntia ja painat Aloita, aika alkaa täydestä. Pelissä näkyy vain peliruutu, ja tulosruudulla pisteet eivät enää muutu.
- ☐ Aloita uusi peli tason 2 jälkeen. Tarkista, että peli alkaa taas tasolta 0.
- ☐ Kirjaa tulokset päiväkirjaan. Jos jokin ei mennyt odotetusti, korjaa, tee commit ja testaa uudelleen.
- ☐ Sulje GitHubissa tämän viikon valmiit issue't. Kirjoita jokaiseen ennen sulkemista kommentti, jossa on linkki commitiin, jossa työ tehtiin.

Valmis kun: Kolmen kierroksen tulokset on kirjattu, ja uusi peli alkaa aina tasolta 0.

Tallenna työnäyte: Testien tulokset ja commit-linkit viikon 39 päiväkirjaan.

Valmis kun: Valinta on perusteltu. Pisteillä 0, 31 ja 61 tilauksen koko tai asiakkaan odotusaika muuttuu tasojen mukaan joka kerta, eikä jokaiselle tasolle tarvita omaa if-lausetta.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: vertailu ja valinta päiväkirjassa, vaikeustasojen commit ja testitulokset.

Vko 40 · 28.9.–2.10. – Feature 5 · Top 5 -tuloslista

Viisi parasta tulosta ja nimimerkit säilyvät, vaikka pelin sulkee ja avaa uudelleen.

Tehtävä 1 / 5 · Lisää tulosruutuun nimimerkkikenttä ja Tallenna-painike.

- ☐ Lisää ResultPaneliin syöttökenttä: UI → Input Field - TextMeshPro. Nimeä se NicknameInput.
- ☐ Valitse NicknameInputin alta Text Area → Placeholder ja kirjoita Inspectorin tekstikenttään vihjetekstiksi Nimimerkki.
- ☐ Lisää ResultPaneliin painike SaveScoreButton tekstillä Tallenna tulos.
- ☐ Lisää ResultPaneliin teksti HighScoresText, johon tulee viisi parasta tulosta. Siirrä Scene-ikkunassa tulosruudun tekstit, kenttä ja painikkeet erilleen ja tee HighScoresText viiden rivin korkuiseksi.

Valmis kun: Tulosruudulla näkyvät nimimerkkikenttä, Tallenna tulos -painike ja tyhjä tuloslista.

Tallenna työnäyte: Kuvakaappaus tulosruudusta polkuun project-docs/evidence/week-40/tulosruutu.png, commit ja push. Polku viikon 40 päiväkirjaan.

Tehtävä 2 / 5 · Tee SaveService-skripti, joka tallentaa viisi parasta tulosta.

- ☐ Luo skripti SaveService avun työpohjasta ja liitä se uuteen tyhjään objektiin SaveService.
- ☐ Lue työpohjasta kaksi luokkaa: ScoreEntry on yksi tulos (nimimerkki ja pisteet) ja ScoreList on tulosten lista.
- ☐ Täydennä SaveScore: lisää uusi tulos listaan ja järjestä lista pisteiden mukaan suurimmasta pienimpään. Poista sitten kuudes tulos ja sitä huonommat.
- ☐ Etsi SaveScore-metodista valmiit tallennusrivit ja kirjoita päiväkirjaan omin sanoin, mitä JsonUtility.ToJson, PlayerPrefs.SetString ja PlayerPrefs.Save tekevät.
- ☐ Täydennä LoadScores: lue teksti PlayerPrefs.GetString-metodilla ja muuta se takaisin listaksi. Jos tallennusta ei vielä ole, palauta tyhjä lista.

Valmis kun: SaveScore ja LoadScores on täydennetty, ja päiväkirjassa on omin sanoin selitys tallennusriveistä.

Tallenna työnäyte: Commit ja push. Commit-linkki ja selitys viikon 40 päiväkirjaan.

Tehtävä 3 / 5 · Kytke tulosruutu tallentamaan ja näyttämään parhaat tulokset.

- ☐ Lisää GameManageriin avun työpohjan kentät nicknameInput, highScoresText ja saveService.
- ☐ Raahaa Inspectorissa NicknameInput, HighScoresText ja SaveService-objekti GameManager-skriptin kenttiin NicknameInput, High Scores Text ja Save Service.
- ☐ Lisää GameManageriin avun ShowHighScores-metodi. Se kirjoittaa listan rivit HighScoresText-tekstiin.
- ☐ Täydennä avun SaveCurrentScore-metodi: se tallentaa nimimerkin ja pisteet SaveServicellä ja näyttää palautetun listan.
- ☐ Kytke SaveScoreButtonin On Click () -listaan GameManager → SaveCurrentScore.
- ☐ Täydennä EndGame-metodia: näytä tallennettu lista heti, kun tulosruutu avautuu.
- ☐ Paina Play ja tallenna kuusi eri tulosta. Tarkista, että lista näyttää viisi parasta suurimmasta pienimpään.

Valmis kun: Tulosruutu näyttää tallennetut tulokset heti, ja kuudesta tuloksesta näkyy viisi parasta oikeassa järjestyksessä.

Tallenna työnäyte: Commit ja push. Kuvakaappaus listasta kansioon project-docs/evidence/week-40/ ja polku viikon 40 päiväkirjaan.

Tehtävä 4 / 5 · Tarkista nimimerkki ennen tallennusta.

- ☐ Päätä nimimerkin enimmäispituus, esimerkiksi 12 merkkiä. Kirjaa päätös päiväkirjaan.
- ☐ Lisää avun CleanNickname-metodi SaveService-skriptiin ja kutsu sitä SaveScore-metodin ensimmäisellä rivillä. Metodi poistaa jo valmiiksi välilyönnit alusta ja lopusta (Trim).
- ☐ Jos nimimerkki on tyhjä, käytä nimeä Nimetön.
- ☐ Jos nimimerkki on liian pitkä, lyhennä se sallittuun pituuteen.
- ☐ Aseta NicknameInput-kentän Character Limit -arvoksi sama enimmäispituus.
- ☐ Kirjaa päiväkirjaan, mitä peli tallentaa (nimimerkki ja pisteet) ja mitä se ei tallenna (oikea nimi, salasanat tai muut henkilötiedot).

Valmis kun: Tyhjä nimi tallentuu nimellä Nimetön, ja liian pitkä nimi lyhenee sallittuun pituuteen.

Tallenna työnäyte: Commit ja push. Päätökset ja commit-linkki viikon 40 päiväkirjaan.

Tehtävä 5 / 5 · Testaa tallennus selaimessa ja perustele ratkaisu.

- ☐ Kirjoita päiväkirjaan odotettu tulos kolmelle testille ennen kuin testaat.
- ☐ Testi 1: tallenna kuusi eri tulosta. Vain viisi parasta näkyy.

- ☐ Testi 2: tallenna kaksi yhtä suurta tulosta. Tarkista, että järjestys noudattaa asiakkaan kanssa sovittua sääntöä. Jos sääntöä ei ole sovittu, aiempi tulos on ylempänä, ja asia on avoin.
- ☐ Testi 3: kopioi pelin osoite, sulje välilehti ja avaa sama osoite uudelleen samassa selaimessa ilman uutta buildia. Sama lista näkyy.
- ☐ Kirjoita päiväkirjan Miksi tein näin? -kenttään, miksi PlayerPrefs sopii tähän peliin. Kirjoita myös, että tiedot säilyvät vain tässä selaimessa ja että pelaaja voi itse muuttaa tai poistaa ne.
- ☐ Sulje GitHubissa tämän viikon valmiit issue't. Kirjoita jokaiseen ennen sulkemista kommentti, jossa on linkki commitiin, jossa työ tehtiin.

Valmis kun: Kolmen testin tulokset ja ratkaisun perustelu ovat päiväkirjassa.

Tallenna työnäyte: Testitulokset, perustelu ja commit-linkit viikon 40 päiväkirjaan.

Valmis kun: Kuudesta tuloksesta näkyy vain viisi parasta myös silloin, kun sivu avataan uudelleen. Tasapisteet ja liian pitkä nimimerkki on testattu.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: tallennuksen commitit, neljän testin tulokset ja ratkaisun perustelu päiväkirjassa.

Vko 41 · 5.–9.10. – Katselmointi · Asiakas pelaa peliäsi

Asiakas pelaa peliä ensimmäistä kertaa ja antaa palautetta. Yhdessä sovitaan yksi muutos.

Tehtävä 1 / 3 · Valmistele asiakkaalle kokeiltava versio.

- ☐ Käy läpi tarkistuslista. Tarkista, että valikko, 1–3 tuotteen tilaus, toimitus, pisteet, ajastin, vaikeus, tulosruutu ja tallennus toimivat.
- ☐ Tee selainversio (Build And Run) ja pelaa se kerran itse alusta loppuun.
- ☐ Kirjoita muistiin version tunniste: päivämäärä ja viimeisimmän commitin tunniste.
- ☐ Sovi ohjaajan kanssa katselmoinnin aika. Katselmointi on tapaaminen, jossa asiakas kokeilee keskeneräistä peliä ja antaa palautetta.
- ☐ Valmistele viiden minuutin esittely: mitä peli tekee, mikä on valmista ja mikä vielä puuttuu.

Valmis kun: Selainversio toimii alusta loppuun, ja katselmoinnin aika on sovittu.

Tallenna työnäyte: Version tunniste ja tarkistuslistan tulos viikon 41 päiväkirjaan.

Tehtävä 2 / 3 · Anna asiakkaan pelata ja kirjaa, mitä hän sanoo.

- ☐ Pyydä asiakasta pelaamaan peli alusta loppuun. Älä neuvo, ellei hän pyydä apua.
- ☐ Kirjoita ylös asiakkaan sanat sellaisinaan, esimerkiksi ”En huomannut tilausta”.
- ☐ Kirjoita erikseen, mitä itse näit: missä asiakas epäröi tai painoi väärää painiketta.
- ☐ Selitä yksi tekninen ratkaisu arkikielellä, esimerkiksi miten tulokset tallentuvat. Kirjaa, ymmärsikö asiakas.
- ☐ Kysy lopuksi: mikä yksi asia pitäisi muuttaa ensin?

Valmis kun: Muistiinpanoissa ovat asiakkaan sanat ja omat havaintosi erikseen.

Tallenna työnäyte: Muistiinpanot viikon 41 päiväkirjaan.

Tehtävä 3 / 3 · Sovi asiakkaan kanssa yksi muutos ja tee siitä issue.

- ☐ Valitse palautteesta yhdessä asiakkaan kanssa yksi muutos, jonka ehdit tehdä viikolla 43.
- ☐ Tee muutoksesta GitHub-issue: otsikko verbillä, asiakkaan alkuperäinen palaute, arvio ja Valmis kun -ehto.
- ☐ Jos asiakas pitää muutosta välttämättömänä, se on pakollinen (P0): anna label P0 pakollinen. Muuten se on tärkeä (P1): anna label P1 tärkeä.
- ☐ Pyydä asiakkaalta hyväksyntä issuelle. Kirjaa hyväksyjän rooli ja päivä.
- ☐ Vertaa kolmen viikolla 35 tehdyn issuen arviota siihen, kauanko työ oikeasti kesti. Kirjaa ero ja sen syy päiväkirjaan.
- ☐ Kirjoita katselmointimuistio päiväkirjaan: päivä, osallistujien roolit, asiakkaan sanat, oma tulkinta ja päätös.

Valmis kun: GitHubissa on yksi hyväksytty muutosissue, ja katselmointimuistio on päiväkirjassa.

Tallenna työnäyte: Issuen linkki ja katselmointimuistio viikon 41 päiväkirjaan.

Valmis kun: Asiakas on pelannut pelin alusta loppuun. Muistiossa näkyvät asiakkaan sanat, oma tulkinta, päätös, hyväksyjä ja yksi rajattu issue.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: version tunniste, katselmointimuistio ja hyväksytty muutosissue.

Vko 42 · 12.–16.10. – Syysloma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 43 viimeisimmästä toimivasta main-versiosta.

Vaihe C – Peli valmiiksi: palautemuutos, käytettävyys, testaus ja koodin laatu

Vko 43 · 19.–23.10. – Feature 6 · Asiakkaan toivoma parannus

Asiakkaan pyytämä muutos on pelattavana. Vanhat ominaisuudet toimivat edelleen.

Tehtävä 1 / 3 · Tarkista muutosissue ja kirjoita sille testi.

- ☐ Avaa viikolla 41 tekemäsi muutosissue GitHubissa ja lue asiakkaan alkuperäinen palaute.
- ☐ Arvioi, onko muutos enintään yhden päivän työ. Jos se on isompi, jaa se kahdeksi issueksi.
- ☐ Kirjoita issueen hyväksymistesti: mitä asiakas tekee ja mitä hän näkee, kun muutos on valmis.
- ☐ Kirjoita issueen myös vanhan pelipolun testi: Aloita → toimita tilaus → tulosruutu toimii kuten ennen.

Valmis kun: [Issuessa ovat arvio, hyväksymistesti ja vanhan pelipolun testi.](#)

Tallenna työnäyte: Issuen linkki viikon 43 päiväkirjaan.

Tehtävä 2 / 3 · Tee muutos omassa Git-haarassa.

- ☐ Varmista ensin, että main-haara eli pääversio toimii: pelaa yksi kierros.
- ☐ Luo uusi haara eli branch: valitse GitHub Desktopissa Current Branch → New Branch. Anna nimeksi esimerkiksi feature/suurempi-tilaus.
- ☐ Tee muutos pienissä osissa. Tee commit aina, kun yksi osa toimii.
- ☐ Testaa jokaisen commitin jälkeen, että tilaus, toimitus ja pisteet toimivat edelleen.
- ☐ Tee push, jolloin haara näkyy GitHubissa. Uuden haaran ensimmäisessä pushissa GitHub Desktopin painike on nimeltään Publish branch.

Valmis kun: [Haara näkyy GitHubissa, ja siinä on vähintään kaksi pientä committia.](#)

Tallenna työnäyte: Haaran nimi ja commit-linkit viikon 43 päiväkirjaan.

Tehtävä 3 / 3 · Pyydä katselmointi pull requestilla ja yhdistä muutos.

- ☐ Avaa repository GitHubissa ja paina Compare & pull request. Jos painiketta ei näy, valitse Pull requests → New pull request ja vertailtavaksi oma haarasi. Pull request on pyyntö yhdistää oma haara main-haaraan.
- ☐ Kirjoita pull requestiin, mitä muutit ja miten testasit. Lisää rivi Closes #numero, jossa numero on muutosissuen numero. Se sulkee issuen, kun pull request yhdistetään.
- ☐ Pyydä ohjaajaa tai vertaista kommentoimaan. Vastaa jokaiseen kommenttiin tai korjaa koodi samaan haaraan ja tee push: korjaus näkyy pull requestissa itsestään.
- ☐ Aja hyväksymistesti ja vanhan pelipolun testi vielä kerran.
- ☐ Paina Merge pull request ja Confirm merge, jolloin muutos siirtyy main-haaraan. Vaihda sitten GitHub Desktopissa haaraksi main ja paina Fetch origin ja Pull origin, jotta koneesi main on ajan tasalla.
- ☐ Kirjaa päiväkirjaan ketju: palaute → issue → haara → pull request → commit → testi. Lisää jokaisesta linkki.

Valmis kun: [Pull request on yhdistetty, siinä on kommentti ja vastaus, ja molemmat testit menivät odotetusti.](#)

Tallenna työnäyte: Pull requestin linkki ja koko ketju viikon 43 päiväkirjaan.

Valmis kun: [Muutos täyttää Valmis kun -ehdon, vanha pelipolku toimii, katselmointikommenttiin on vastattu ja muutos on yhdistetty main-haaraan.](#)

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: issue, haara, pull request kommentteineen, commitit ja testitulokset.

Vko 44 · 26.–30.10. – Feature 7 · Peli ohjaa pelaajaa itse

Uusi pelaaja ymmärtää tavoitteen ja pelaa kierroksen ilman, että kukaan neuvoo vieressä.

Tehtävä 1 / 3 · Vertaa peliä viikon 35 luonnokseen.

- ☐ Avaa viikon 35 luonnos `project-docs/evidence/week-35/mockup.png`.
- ☐ Ota kuvakaappaus pelisi valikosta, peliruudusta ja tulosruudusta.
- ☐ Kirjoita jokaisesta ruudusta, mikä on erilaista kuin luonnoksessa.
- ☐ Kirjoita jokaisen eron perään, miksi muutit sen, tai merkitse se korjattavaksi.

Valmis kun: Jokaisesta kolmesta ruudusta on kirjattu erot ja niiden syyt.

Tallenna työnäyte: Kuvakaappaukset kansioon `project-docs/evidence/week-44/`, commit ja push. Vertailu viikon 44 päiväkirjaan.

Tehtävä 2 / 3 · Testaa peliä uudella pelaajalla ilman neuvoja.

- ☐ Pyydä kokeilijaksi vertainen, joka ei ole pelannut peliäsi.
- ☐ Anna hänelle yksi tehtävä: ”Aloita peli, toimita yksi tilaus ja katso tuloksesi.” Älä neuvo.
- ☐ Kirjaa havainnot: missä hän epäröi, mitä hän painoi väärin ja mitä hän kysyi.
- ☐ Mittaa, kauanko aloituksesta kestää ensimmäiseen oikeaan toimitukseen.

Valmis kun: Päiväkirjassa ovat testajan rooli ja päivä, annettu tehtävä, havainnot ja aika ensimmäiseen toimitukseen.

Tallenna työnäyte: Havainnot viikon 44 päiväkirjaan.

Tehtävä 3 / 3 · Korjaa kaksi suurinta ongelmaa ja testaa uudelleen.

- ☐ Valitse havainnoista kaksi, jotka haittasivat pelaamista eniten.
- ☐ Ota kuvakaappaus molemmista kohdista ennen muutosta.
- ☐ Tee korjaukset, esimerkiksi isompi teksti, selkeämpi painike tai lyhyt ohjeteksti.
- ☐ Ota kuvakaappaus molemmista kohdista muutoksen jälkeen. Tee commit ja push.
- ☐ Anna sama tehtävä toiselle uudelle pelaajalle ja kirjaa, autoivatko korjaukset.
- ☐ Sulje GitHubissa tämän viikon valmiit issue't. Kirjoita jokaiseen ennen sulkemista kommentti, jossa on linkki commitiin, jossa työ tehtiin.

Valmis kun: Kahdesta korjauksesta on kuvat ennen korjausta ja sen jälkeen, ja uusintatestin tulos on kirjattu.

Tallenna työnäyte: Ennen/jälkeen-kuvat kansioon `project-docs/evidence/week-44/`. Uusintatestin tulos ja commit-linkki viikon 44 päiväkirjaan.

Valmis kun: Uusi pelaaja ymmärtää tavoitteen ja pelaa yhden tilauksen loppuun ilman neuvoja. Kahdesta korjauksesta on kuvat ennen korjausta ja sen jälkeen.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: vertailu luonnokseen, havainnot, ennen/jälkeen-kuvat ja uusintatestin tulos.

Vko 45 · 2.–6.11. – Pelin laatu · Peli kestää pelaamista

Peli ei kaadu rajatilanteissa. Ajan loppuminen, kaksoispainallukset ja rikottu tuotelista on testattu.

Tehtävä 1 / 3 · Kirjoita 12 testitapausta ennen testaamista.

- ☐ Luo tiedosto `project-docs/testaus.md` avun taulukkopohjasta.
- ☐ Numeroi testitapaukset T01, T02, T03 ja niin edelleen. T tarkoittaa testitapausta ja numero sen järjestystä. Tunnuksella viittaat testiin myöhemmin.
- ☐ Kirjoita neljä testitapausta tavalliselle pelaamiselle: aloitus, oikea toimitus, väärä toimitus ja uusi peli.
- ☐ Kirjoita neljä testitapausta rajatilanteille: aika 0, kaksoispainallus, tyhjä nimimerkki ja liian pitkä nimimerkki.
- ☐ Kirjoita neljä testitapausta datalle eli tiedostoille ja tallennukselle: puuttuva `products.json`, rikkinäinen `products.json`, tyhjä tallennus ja kuudes tulos top 5 -listaan.
- ☐ Täytä jokaiseen testitapaukseen sarakkeet Lähtötila, Mitä teen ja Odotettu tulos ennen kuin ajat yhtään testiä. Käytä omia arvojasi: kierroksen pituus, väärän toimituksen pisteet ja nimimerkin enimmäispituus.

Valmis kun: Tiedostossa on 12 testitapausta T01–T12, ja jokaisella on odotettu tulos.

Tallenna työnäyte: `project-docs/testaus.md`, `commit` ja `push`.

Tehtävä 2 / 3 · Aja testitapaukset selainversiolla ja kirjaa tulokset.

- ☐ Tee selainversio (Build And Run) ja kirjaa version tunniste testaustaulukon alkuun.
- ☐ Aloita puhtaasta tilanteesta: avaa peli yksityisessä selainikkunassa, jolloin vanhat tulokset eivät ole mukana.
- ☐ Aja testitapaukset T01–T12 järjestyksessä. Kirjaa jokaisesta, mitä tapahtui ja läpäisikö testi.
- ☐ Tyhjennä datatestejä varten `Products Json` -kenttä tai poista `products.json`-tiedostosta pilkku. Tee uusi selainversio ja palauta kenttä tai tiedosto testin jälkeen.
- ☐ Älä merkitse testiä läpäistykseksi, jos et ajanut sitä itse.

Valmis kun: Jokaisella testitapauksella T01–T12 on todellinen tulos ja merkintä, läpäisikö se.

Tallenna työnäyte: Päivitetty `project-docs/testaus.md`, `commit` ja `push`.

Tehtävä 3 / 3 · Korjaa kolme virhettä ja kirjaa jokaisesta koko ketju.

- ☐ Valitse kolme testitapausta, jotka eivät läpäisseet. Jos aitoja virheitä ei ole kolmea, pyydä ohjaajalta vikatehtävä eli ohjaajan valmisteleva virhe, jonka etsit ja korjaat.
- ☐ Kirjaa jokaisesta, miten virhe toistetaan, mikä oli odotettu tulos ja mitä tapahtui.
- ☐ Etsi syy ennen kuin korjaat. Kirjoita syy yhdellä virkkeellä.
- ☐ Korjaa ja tee `commit`, jonka viestissä on testitapauksen tunnus. Esimerkiksi: "Korjaa testitapaus T05, tulosruutu aukeaa vain kerran."
- ☐ Aja sama testitapaus uudelleen. Aja myös regressiotesti eli toinen testitapaus, joka käyttää samaa koodia: se näyttää, ettei korjaus rikkonut muuta.
- ☐ Kirjaa jokainen ketju päiväkirjaan: havainto → toistamisohje → syy → korjauscommit → uusintatesti → regressiotesti.

Valmis kun: Päiväkirjassa on kolme täydellistä ketjua, ja jokaisen uusintatesti meni odotetusti.

Tallenna työnäyte: Kolme ketjua `commit`-linkkeineen viikon 45 päiväkirjaan.

Valmis kun: Kaikissa 12 testitapauksessa näkyvät lähtötila, toiminta, odotettu tulos, todellinen tulos ja läpäisy. Kolmessa ketjussa näkyvät havainto, syy, korjauscommit ja onnistunut uusintatesti.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: `project-docs/testaus.md`, kolme virheenkorjausketjua ja korjauscommitit.

Vko 46 · 9.–13.11. – Pelin laatu · Koodi, jonka toinen ymmärtää

Peli toimii kuten ennen. Koodi on selkeämpi, ja osaat selittää ratkaisusi.

Tehtävä 1 / 3 · Etsi koodista yksi kohta, jota on vaikea ymmärtää.

- ☐ Avaa C#-skriptit ja etsi yksi näistä: sama koodi kahdessa paikassa, nimi, joka ei kerro tehtäväänsä (esimerkiksi a tai temp) tai yli 30 rivin metodi.
- ☐ Etsi myös skripti, joka tekee monta eri asiaa, esimerkiksi laskee pisteet ja päivittää tekstiä.
- ☐ Valitse näistä yksi kohta. Kirjoita päiväkirjaan, mikä kohdassa on vaikeaa ja miksi.
- ☐ Valitse testaustaulukosta testitapaus, joka kulkee tämän koodin kautta. Kirjaa testitapausten tunnus, esimerkiksi T03.

Valmis kun: Päiväkirjassa on yksi valittu kohta, sen ongelma ja siihen liittyvä testitapaus.

Tallenna työnäyte: Kuvaus ja testitapausten tunnus viikon 46 päiväkirjaan.

Tehtävä 2 / 3 · Selkeytä kohta niin, että peli toimii kuten ennen.

- ☐ Aja valitsemasi testitapaus ennen muutosta ja kirjaa tulos.
- ☐ Tee yksi rajattu muutos, esimerkiksi anna metodille kuvaava nimi tai siirrä toistuva koodi omaan metodiinsa.
- ☐ Aja sama testitapaus muutoksen jälkeen. Tuloksen pitää olla sama kuin ennen.
- ☐ Tee commit, jonka viesti kertoo muutoksen, esimerkiksi "Selkeytä pisteiden laskua: CalculateOrderScore".

Valmis kun: Sama testitapaus läpäisee ennen ja jälkeen muutoksen, ja muutos on yhdessä commitissa.

Tallenna työnäyte: Commit-linkki sekä ennen- ja jälkeen-tulokset viikon 46 päiväkirjaan.

Tehtävä 3 / 3 · Pyydä koodikatselmointi ja selitä ratkaisusi.

- ☐ Pyydä ohjaajaa tai vertaista lukemaan muuttamasi koodi. Koodikatselmointi tarkoittaa, että toinen ihminen lukee koodin ja kommentoi sitä.
- ☐ Näytä hänelle koodi ennen ja jälkeen muutoksen, esimerkiksi GitHubin commit-näkymästä. Kirjaa saamasi kommentti sellaisenaan.
- ☐ Vastaa kommenttiin: korjaa koodia tai perustele, miksi pidät ratkaisun.
- ☐ Selitä samalla tapaamisella suullisesti yksi kohta, jossa käytit tekoälyä apuna. Jos et käyttänyt tekoälyä, selitä avun työpohjasta otettu kohta.
- ☐ Tarkista, että tekoälyn käyttö on kirjattu AI-lokiin eli sivuston AI-loki-näkymään, johon kirjaat tekoälyn avun.

Valmis kun: Päiväkirjassa ovat katselmoijan rooli, hänen kommenttinsa, oma vastauksesi ja se, minkä kohdan selität suullisesti.

Tallenna työnäyte: Kommentti, vastaus ja selitetty kohta viikon 46 päiväkirjaan. Tekoälyn käyttö AI-lokiin.

Valmis kun: Sama testitapaus läpäisee ennen ja jälkeen refaktoroinnin. Katselmointikommenttiin on vastattu, ja osaat selittää ratkaisusi ilman tekoälyä.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: refaktoroinnin commit, testi ennen ja jälkeen, katselmoinnin kommentti ja vastaus.

Vaihe D – Julkaisu ja näyttö

Vko 47 · 16.–20.11. – Pelin julkaisu · Ensimmäinen julkaisuehdokas RC1 testipelattavana

Koko peli on pelattavana täsmälleen siinä muodossa, jossa se julkaistaan. Uusia ominaisuuksia ei enää lisätä.

Tehtävä 1 / 3 · Merkitse ensimmäinen julkaisuehdokas (RC1) ja lopeta uusien ominaisuuksien lisääminen.

- ☐ Kirjaa päiväkirjaan päivä, josta alkaen peliin ei lisätä uusia ominaisuuksia vaan korjataan vain virheitä. Tätä kutsutaan ominaisuusjäädätykseksi.
- ☐ Pelaa main-haaran versio läpi ja varmista, että valikko, peliruutu, tulosruutu ja tallennus toimivat.
- ☐ Tarkista, että viimeisin commit on pushattu GitHubiin: merkitset sen seuraavassa osassa tagilla RC1. RC1 tarkoittaa ensimmäistä julkaisuehdokasta (release candidate 1): versiota, joka julkaistaan, jos testeissä ei löydy vakavia virheitä.
- ☐ Avaa GitHubissa Releases → Draft a new release. Kirjoita Choose a tag -kenttään RC1 ja valitse Create new tag. Rastita Set as a pre-release ja paina Publish release.
- ☐ Tee julkaisuehdokkaasta (RC1) selainversio Build And Run -toiminnolla. Testaajat pelaavat sitä sinun koneellasi.

Valmis kun: Tagi RC1 näkyy GitHubissa, ja se osoittaa main-haaran viimeisimpään committiin.

Tallenna työnäyte: RC1-tagin linkki ja commitin tunniste viikon 47 päiväkirjaan.

Tehtävä 2 / 3 · Anna asiakkaan ja yhden muun ihmisen testata julkaisuehdokas.

- ☐ Pyydä asiakasta ja yhtä muuta ihmistä testaamaan julkaisuehdokas (RC1).
- ☐ Anna molemmille sama tehtävä: ”Pelaa peli alusta loppuun ja tallenna tulos.”
- ☐ Pyydä heitä sulkemaan peli ja avaamaan se uudelleen, jotta näet, toimiiko tallennus.
- ☐ Kirjaa jokainen havainto erikseen: kuka testasi (rooli), mitä tapahtui ja missä kohdassa.

Valmis kun: Kahden testaajan havainnot on kirjattu erikseen, ja testaajien roolit näkyvät.

Tallenna työnäyte: Havainnot viikon 47 päiväkirjaan.

Tehtävä 3 / 3 · Päätä jokaisesta havainnosta, korjataanko se ennen julkaisua.

- ☐ Kirjoita jokainen havainto omalle rivilleen.
- ☐ Merkitse vakavuus: vakava (peli ei toimi tai kaatuu), haitallinen (peli toimii mutta hankalasti) tai pieni (ulkonäköasia).
- ☐ Merkitse toistuvuus: toistuu aina, joskus tai kerran.
- ☐ Päätä jokaisesta: korjataan nyt, tunnettu puute (kerrotaan käyttöohjeessa) tai myöhemmin (parannus, joka ei haittaa pelaajaa).
- ☐ Tee jokaisesta korjattavasta havainnosta GitHub-issue. Lisää sille uusi testitapaus project-docs/testaus.md-tiedostoon, esimerkiksi testitapaus T13.

Valmis kun: Jokaisella havainnolla on vakavuus, toistuvuus ja päätös, ja korjattavista on issueet.

Tallenna työnäyte: Luokiteltu lista ja issueiden linkit viikon 47 päiväkirjaan.

Valmis kun: Ensimmäinen julkaisuehdokas (RC1) on merkitty yhteen committiin. Asiakas ja toinen käyttäjä ovat testanneet sen, ja jokaisella havainnolla on vakavuus, toistuvuus ja päätös.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: RC1-tagin, kahden testaajan havainnot ja luokiteltu korjauslista.

Vko 48 · 23.–27.11. – Pelin julkaisu · v1.0 julki

Peli on julkaistu. Linkki toimii muillakin koneilla, ja asiakas voi kokeilla peliä itse.

Tehtävä 1 / 4 · Korjaa vain ne virheet, jotka päätit korjata ennen julkaisua.

- ☐ Avaa viikon 47 luokiteltu lista. Ota työn alle vain havainnot, joiden päätös on "korjataan nyt".
- ☐ Korjaa yksi virhe kerrallaan ja tee jokaisesta oma commit.
- ☐ Aja jokaisen korjauksen jälkeen siihen liittyvä testitapaus ja regressiotesti eli toinen testitapaus, joka käyttää samaa koodia.
- ☐ Sulje korjatut GitHub-issuet ja lisää niihin linkki korjauscommittiin.

Valmis kun: Kaikki korjataan nyt -havainnot on korjattu ja testattu, ja niiden issuet on suljettu.

Tallenna työnäyte: Korjauscommitien linkit viikon 48 päiväkirjaan.

Tehtävä 2 / 4 · Julkaise peli GitHub Pagesiin ja merkitse se versioksi v1.0.

- ☐ Avaa File → Build Profiles, valitse Web ja tarkista, että CafeGame on scene-listassa.
- ☐ Avaa Player Settings → Web → Publishing Settings ja ota käyttöön Decompression Fallback. Ilman sitä GitHub Pages ei osaa avata pakattua peliä.
- ☐ Paina Build ja valitse kansio repositoryn docs-kansio. Luo VS Codella docs-kansioon tyhjä tiedosto .nojekyll: se kertoo GitHub Pagesille, että kansio julkaistaan sellaisenaan.
- ☐ Tee commit ja push. Tarkista GitHubista, että docs-kansiossa näkyvät index.html, Build ja TemplateData.
- ☐ Avaa GitHubissa Settings → Pages. Valitse Deploy from a branch, haaraksi main ja kansioksi /docs. Paina Save.
- ☐ Odota muutama minuutti, avaa Pages-sivun antama linkki ja pelaa yksi kierros.
- ☐ Kun peli aukeaa linkistä, merkitse sama commit tagilla v1.0: Releases → Draft a new release → tagi v1.0 → Publish release.

Valmis kun: Peli aukeaa julkaisulinkistä, ja tagi v1.0 osoittaa samaan committiin kuin julkaisu.

Tallenna työnäyte: Julkaisulinkki ja v1.0-tagin linkki viikon 48 päiväkirjaan.

Tehtävä 3 / 4 · Kirjoita README:hen käyttöohje ja tunnetut puutteet.

- ☐ Lisää README.md-tiedostoon otsikko "Näin pelaat". Lisää sen alle julkaisulinkki.
- ☐ Kirjoita ohje neljänä vaiheena: avaa linkki → Aloita → valitse tilauksen tuotteet → Toimita.
- ☐ Lisää otsikko "Tunnetut puutteet" ja listaa sen alle viikon 47 tunnetut puutteet.
- ☐ Lisää otsikko "Lähteet ja lisenssit". Listaa sen alle käytetyt grafiikkapaketit ja niiden lisenssit.
- ☐ Pyydä toista ihmistä aloittamaan peli pelkän ohjeen avulla, ilman suullista apua. Kirjaa hänen roolinsa ja kohdat, joissa hän epäröi, ja korjaa ne ohjeeseen.

Valmis kun: README:ssä ovat julkaisulinkki, käyttöohje, tunnetut puutteet ja lähteet.

Tallenna työnäyte: README.md, commit ja push.

Tehtävä 4 / 4 · Testaa julkaistu peli toisella laitteella käyttöohjeen avulla.

- ☐ Pyydä toista ihmistä avaamaan julkaisulinkki eri laitteella tai eri selaimella.
- ☐ Pyydä häntä pelaamaan kierros README:n käyttöohjeen avulla aloituksesta tulosruutuun ja tallentamaan tulos.
- ☐ Pyydä häntä päivittämään sivu ja tarkistamaan, että tulos näkyy yhä.
- ☐ Kirjaa testattu laite ja selain, testaajan rooli ja tulos.

Valmis kun: Toinen ihminen on pelannut julkaistun pelin toisella laitteella tai selaimella, ja tulos on kirjattu.

Tallenna työnäyte: Laite, selain, testaajan rooli ja tulos viikon 48 päiväkirjaan.

Valmis kun: v1.0-tagin commit vastaa julkaistua versiota. Toinen ihminen avaa linkin toisella selaimella tai laitteella ja pelaa kierroksen käyttöohjeen avulla.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: julkaisulinkki, v1.0-tag, testitulos toiselta laitteelta ja README käyttöohjeineen.

Vko 49 · 30.11.–4.12. – Näyttö · Aineisto ja luovutus

Peli, repository ja projektipäiväkirja todistavat osaamisesi ilman suullista selitystä.

Tehtävä 1 / 5 · Tarkista projektipäiväkirja ja vie se repositoryyn.

- ☐ Avaa Projektipäiväkirja-näkymä ja tarkista, että jokainen viikko on merkitty kirjatuksi.
- ☐ Täydennä puuttuvat kentät viikkonäkymissä.
- ☐ Tarkista jokaisen viikon Missä työnäyte on? -kentästä, että linkki aukeaa.

Valmis kun: Kaikki 15 viikkoa on kirjattu, ja jokaisen viikon työnäytelinkki aukeaa.

Tallenna työnäyte: Täydennetyt viikkomerkinnot sivuston projektipäiväkirjassa. Lataat tiedoston repositoryyn tehtävässä 5.

Tehtävä 2 / 5 · Kirjoita itsearviointi omiin työnäytteisiin nojaten.

- ☐ Valitse projektista kolme asiaa, jotka osaat nyt hyvin.
- ☐ Liitä jokaiseen vahvuuteen työnäyte: linkki commitiin, testitapaukseen tai päiväkirjan viikkoon.
- ☐ Kirjoita yksi asia, jota haluat kehittää seuraavaksi. Kirjoita myös, miten aiot kehittää sitä.
- ☐ Kirjoita itsearviointi itse. Älä käytä tekoälyä tekstin kirjoittamiseen.

Valmis kun: Viikon 49 päiväkirjassa on kolme vahvuutta työnäytteineen ja yksi kehitysaskel.

Tallenna työnäyte: Itsearviointi viikon 49 päiväkirjaan.

Tehtävä 3 / 5 · Liitä jokaiseen näyttömatriisiin kohtaan tarkka linkki.

- ☐ Avaa Näyttömatriisi-näkymä eli luettelo osaamisvaatimuksista, joihin tarvitset työnäytteen, ja lue ensimmäinen vaatimus.
- ☐ Etsi työnäyte, joka osoittaa, että vaatimus täyttyy: issue, C#-tiedosto, commit, testitapaus tai päiväkirjan viikko.
- ☐ Kopioi työnäytteen tarkka linkki. Linkin pitää avata juuri se kohta, ei repositoryn etusivua.
- ☐ Kirjoita viikon 49 päiväkirjaan omalle rivilleen vaatimuksen nimi ja linkki. Rastita sitten vaatimus Näyttömatriisi-näkymässä.
- ☐ Toista osatehtävät 1–4 jokaiselle vaatimukselle.

Valmis kun: Jokaisella näyttömatriisin vaatimuksella on tarkka linkki, joka aukeaa.

Tallenna työnäyte: Vaatimusten linkit viikon 49 päiväkirjaan, josta ne tulevat mukaan projektipäiväkirja.md-tiedostoon.

Tehtävä 4 / 5 · Harjoittele 8–10 minuutin demo.

- ☐ Kirjoita demon runko kuutena kohtana: pelin kulku, tuotelista JSON-tiedostossa, tallennus, yksi virheenkorjaus, Git-historia ja AI-loki.
- ☐ Harjoittele demo kerran ääneen toiselle ihmiselle ja ota aika. Kirjaa kuulijan rooli ja yksi hänen palautteensa. Tavoite on 8–10 minuuttia.
- ☐ Harjoittele, miten selität yhden C#-metodin omin sanoin.
- ☐ Varmista, että demossa käytät julkaistua versiota v1.0.

Valmis kun: Demo kestää 8–10 minuuttia ja käy läpi kaikki kuusi kohtaa.

Tallenna työnäyte: Demon runko viikon 49 päiväkirjaan.

Tehtävä 5 / 5 · Luovuta peli ja aineisto viimeistään pe 4.12.2026.

- ☐ Paina Projektipäiväkirja-näkymän painiketta Lataa koko päiväkirja (.md) ja korvaa repositoryn tiedosto project-docs/projektipäiväkirja.md. Tee commit ja push. Tarkista, että repositoryssa ovat peli, README.md, gdd.md, testaus.md ja projektipäiväkirja.md.
- ☐ Avaa ladattu projektipäiväkirja.md ja tarkista, että sen lopussa on otsikko AI-loki ja omat merkintäsi.
- ☐ Pyydä toista ihmistä avaamaan julkaisulinkki ja repository. Kirjaa, löysikö hän kaiken.
- ☐ Luovuta peli, repository, projektipäiväkirja ja näyttöaineisto ohjaajalle viimeistään pe 4.12.2026.

Valmis kun: Aineisto on luovutettu viimeistään pe 4.12.2026, ja toinen ihminen on tarkistanut sen.

Tallenna työnäyte: Luovutuksen päivä viikon 49 päiväkirjaan.

Valmis kun: Jokaisessa näyttömatriisin kohdassa on tarkka linkki, joka aukeaa. Projektipäiväkirja ja AI-loki ovat repositoryssa, ja demo käyttää samaa v1.0-versiota.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: peli, repository, project-docs/projektipäiväkirja.md, näyttömatriisi, AI-loki ja demo.

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

repository vko 34	<i>Projektin koodivarasto Gitissä.</i> Repository on projektin tiedostojen ja koko muutoshistorian säilytyspaikka. Tässä projektissa se on GitHubissa, ja sieltä löytyvät peliprojekti, project-docs-kansio ja työnäytteet.
commit vko 34	<i>Versionhallintaan tallennettu muutos.</i> Commit on yksi versionhallintaan tallennettu muutostyökalu: mitä muutit ja miksi. Jokaisella commitilla on oma tunnistus, johon voit linkittää työnäytteenä.
build vko 34	<i>Pelin ajettava julkaisuversio.</i> Build on Unityn tekemä valmis versio pelistä. Sitä pelataan ilman Unity-editoria, ja juuri sitä asiakas kokeilee.
WebGL vko 34	<i>Unityn selainjulkaisu.</i> WebGL on Unityn julkaisumuoto, joka toimii selaimessa ilman asennusta. WebGL-build on pelin selainversio – tämän projektin lopputuote.
push vko 34	<i>Commitien siirto GitHubiin.</i> Push siirtää koneellasi tehdyt commitit GitHubiin. Vasta pushin jälkeen ohjaaja ja asiakas näkevät muutoksesi.
P0 vko 34	<i>Pakollinen ydin.</i> P0 on se osa peliä, jonka on pakko valmistua: ilman sitä peliä ei voi luovuttaa asiakkaalle. Tee koko P0 valmiiksi ennen kuin aloitat lisäominaisuuksia.
P1 vko 35	<i>Tärkeä jatkosisältö.</i> P1 on ominaisuus, joka tehdään vasta, kun koko P0 toimii. Se parantaa peliä, mutta peli on luovutettavissa myös ilman sitä.
P2 vko 35	<i>Valinnainen lisä.</i> P2 on ominaisuus, joka voidaan jättää kokonaan pois, jos aika loppuu. Merkitse P2:ksi kaikki, mistä voi luopua ilman, että asiakkaan vaatimus jää täyttämättä.
GDD vko 35	<i>Game Design Document, pelin suunnitteludokumentti.</i> GDD kokoaa yhteen tiedostoon pelin konseptin, pelin kulun, omat suunnittelupäätökset, rajauksen ja avoimet asiat. Tässä projektissa se täytetään Suunnitelma-näkymässä ja tallennetaan repositoryyn nimellä gdd.md.
GitHub-issue vko 35	<i>Tehtävä, jota seurataan GitHubissa.</i> GitHub-issue on yksi tehtävä GitHubissa: otsikko, perustelu, rajattu muutos ja Valmis kun -ehto. Tässä projektissa yhden issuen työmäärä on puolesta päivästä yhteen päivään.
backlog vko 35	<i>Priorisoitu tehtävälista.</i> Backlog on projektin tehtävälista. Tässä projektissa se on GitHubin Issues-lista, jossa jokaisella tehtävällä on prioriteetti: pakollinen (P0), tärkeä (P1) tai lisä (P2). Lista päivitetään aina, kun asiakas päättää jotain uutta.
mockup vko 35	<i>Käyttöliittymän luonnoskuva.</i> Mockup on paperille tai piirto-ohjelmalla tehty luonnos ruuduista ennen koodaamista. Tässä projektissa se on project-docs/evidence/week-35/mockup.png, ja ruudut rakennetaan sen mukaan.
UI vko 35	<i>User interface, käyttöliittymä.</i> Käyttöliittymä on se osa peliä, jonka pelaaja näkee ja jota hän käyttää: painikkeet, tekstit ja paneelit. Unityssä ne rakennetaan Canvas-alueelle.
feature vko 35	<i>Pelin yksittäinen ominaisuus.</i> Feature on yksi pelaajalle näkyvä ominaisuus, esimerkiksi pistelasku tai viiden parhaan tuloksen lista. Tässä projektissa featuret tehdään yksi kerrallaan, yleensä yksi viikossa.
scene vko 34	<i>Unityn näkymä.</i> Scene on Unityn tiedosto, jossa pelin objektit ovat. Tässä projektissa koko peli on yhdessä scenessä nimeltä CafeGame.
Hierarchy vko 36	<i>Unityn objekttilista.</i> Hierarchy-ikkuna näyttää scenen kaikki objektit puuna. Siinä luodaan ja nimetään esimerkiksi Canvas, paneelit, painikkeet ja GameManager.
metodi vko 36	<i>Skriptin toiminto.</i> Metodi on C#-skriptin nimetty toiminto, esimerkiksi StartGame. Painike kutsuu metodia, kun pelaaja klikkaa sitä.
TODO vko 36	<i>Täydennettävä kohta koodissa.</i> TODO-rivi on työpohjaan merkitty kohta, jonka kirjoitat itse. Rivi alkaa merkeillä // TODO, ja sen perässä lukee, mitä kohtaan tulee.
hyväksymistesti vko 36	<i>Testi, joka osoittaa tehtävän valmiiksi.</i> Hyväksymistesti kertoo, mitä tehdään ja mitä pitää näkyä, jotta tehtävä tai muutos on valmis. Esimerkiksi Aloita → Kahvi → Toimita → tulostulokilla 10.

Canvas vko 36	<i>Käyttöliittymän alue Unityssä.</i> Canvas on alue, jolle Unityssä sijoitetaan tekstit, painikkeet ja paneelit. Pelin kolme ruutua ovat Canvasin sisällä.
Inspector vko 36	<i>Unityn asetussikkuna.</i> Inspector näyttää valitun objektin asetukset ja skriptien kentät. Siihen raahataan esimerkiksi paneelit ja products.json-tiedosto.
asset vko 36	<i>Peliin tuotava valmis tiedosto.</i> Asset on peliin tuotava valmis kuva-, ääni- tai fonttitiedosto, esimerkiksi sprite eli hahmon tai esineen kuva. Kirjaa jokaisesta assetista lähde ja lisenssi.
JSON vko 35	<i>Tekstimuoto datalle.</i> JSON on yksinkertainen tekstimuoto, jossa tieto on nimi-arvo-pareina. Tässä projektissa kahvilan tuotteet ovat products.json-tiedostossa, joten valikoimaa voi muuttaa koskematta koodiin.
Console vko 37	<i>Unityn viesti-ikkuna.</i> Console näyttää skriptien Debug.Log- ja Debug.LogError-viestit sekä virheet. Avaa se valinnalla Window → General → Console.
rajatilanne vko 38	<i>Pelin ääritilanne.</i> Rajatilanne on kohta, jossa peli menee helpoimmin rikki, esimerkiksi aika 0, nopea kaksoispainallus tai tyhjä nimimerkki. Rajatilanteet testataan erikseen.
vertainen vko 39	<i>Toinen opiskelija.</i> Vertainen on toinen opiskelija, joka keskustelee kanssasi, testaa peliäsi tai lukee koodiasi. Hän ei ole projektin asiakas.
PlayerPrefs vko 40	<i>Unityn pieni tallennuspaikka.</i> PlayerPrefs tallentaa pieniä tietoja avain-arvo-pareina. Selainversiossa tieto säilyy selaimen muistissa, kunnes pelaaja tyhjentää sen.
katselmointi vko 41	<i>Työn yhteinen tarkastus.</i> Katselmoinnissa toinen ihminen kokeilee peliä tai lukee koodia ja antaa palautetta. Viikolla 41 asiakas katselmoi pelin, viikolla 46 ohjaaja tai vertainen katselmoi koodin.
branch vko 43	<i>Git-haara.</i> Branch eli haara on rinnakkainen kehityslinja: teet muutoksen omassa haarassa, jolloin toimiva main-haara pysyy ehjänä. Valmis haara yhdistetään eli mergetään takaisin main-haaraan.
pull request vko 43	<i>PR, pyyntö yhdistää haara pääversioon.</i> Pull request eli PR on GitHubissa tehtävä pyyntö yhdistää oma haara main-haaraan. Se antaa katselmoinnille oman paikan ennen yhdistämistä. Pienen muutoksen voi myös yhdistää suoraan ilman PR:ää.
T01 vko 45	<i>Testitapauksen tunnus.</i> Testitapaukset numeroidaan juoksevasti: T01 on ensimmäinen testitapaus, T02 toinen. Odotettu tulos kirjataan ennen ajoa, ja tunnuksella viitataan testiin päiväkirjassa ja näyttömatriisissa.
refaktorointi vko 46	<i>Koodin rakenteen selkeyttäminen.</i> Refaktoroinnissa koodia muutetaan selkeämmäksi niin, että peli toimii täsmälleen kuten ennen. Sama testitapaus ajetaan ennen ja jälkeen muutoksen.
regressiotesti vko 45	<i>Vanhan toiminnan uusintatesti.</i> Regressiotesti on toinen testitapaus, joka käyttää samaa koodia kuin korjaus. Kun se menee läpi, tiedät, ettei korjaus rikkonut muuta.
RC vko 47	<i>Release candidate, julkaisuehdokas.</i> Julkaisuehdokas on lähes valmis versio, joka testataan täsmälleen siinä muodossa, jossa se aiotaan julkaista. RC1 on ensimmäinen julkaisuehdokas, eikä siihen enää lisätä uusia ominaisuuksia.
tagi vko 47	<i>Versionhallintaan merkitty nimetty versio.</i> Tagi on yhteen committiin kiinnitetty nimilappu, esimerkiksi RC1 tai v1.0. Tagin avulla löydät myöhemmin täsmälleen sen version, jonka asiakas testasi.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- **Kehitysympäristö** – Editorikuva, build ja repository.
- **Virheenkorjaus** – Kolme täydellistä ketjua aidoista havainnoista tai opettajan merkityistä vikatehtävistä.
- **Toimintojen testaus** – 12 testiä odotettuine ja toteutuneine tuloksineen.
- **Rakenteinen ohjelmointi** – Vastuutettu koodirakenne ja katselmointi.
- **Ylläpidettävä koodi** – Selkeät nimet, pienet metodit ja refaktorointicommit.
- **Käyttöliittymä** – Mockup sekä ennen/jälkeen-kuvat.
- **Toimintojen toteutus** – GitHub-issuet, valmis kun -ehdot ja commitit.
- **Tehtävistä sopiminen** – Tehtävälista ja kokousmuistio.
- **Ratkaisuvaihtoehdot** – Kahden toteutustavan vertailu ja päätös.
- **Ratkaisujen arviointi** – Koodi- tai peliversion katselmointiloki.
- **Oma toiminta tiimissä** – Itsearviointi ja seuraava kehitysaskel.

Ohjelmistokehittäjänä toimiminen

- **Asiakkaan tarpeet** – Kysymyslista ja aloitusmuistio.
- **Asiakaslähtöinen viestintä** – Ymmärrettävä yhteenveto vaihtoehdoista ja rajoista.
- **Version katselmointi** – Lokit viikoilta 41 ja 47.
- **Priorisointi** – Backlog P0-, P1- ja P2-tasoinen ennen ja jälkeen palautteen.
- **Tehtäviksi jakaminen** – 0,5–1 päivän issuet ja nimetyt tekijät.
- **Suunnittelu ja arviointi** – Työmääräarvio verrattuna toteumaan.
- **Toimintalogiikka** – Tilaus, toimitus, pisteet, aika, vaikeus ja pelitilat.
- **Tietovaraston valinta** – JSON- ja tallennusratkaisun perustelu.
- **Yhteys tietovarastoon** – Tuotetietojen luku sekä tuloksen tallennus ja lataus.
- **Rajapinnat ja tieto** – Tiedonkulun kuvaus ja sitä vastaava koodi.
- **Tietoturva** – Syötteiden, tallennuksen ja salaisuuksien riskit sekä ratkaisut.
- **Versionhallinta** – Jatkuva Git-historia ja toimiva main.
- **Osan liittäminen** – Testattu yhdistäminen (merge) tai pull request.
- **Tuotantojulkaisu** – Toimiva v1.0-julkaisulinkki.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- **Kehittäisympäristön käyttöönotto** – Unity Hub, sovittu Unity-versio, 2D-projekti ja WebGL-testibuild.
- **Kirjaston mahdollisuudet ja rajoitteet** – Viikon 40 kirjaus tallennusratkaisun rajoituksista ja viikon 48 julkaisupäätökset tunnettuine puutteineen.
- **Kirjaston toiminnot ja työkalut** – Canvas-käyttöliittymä, Inspector-kytkennät, Play Mode ja Build Profiles committeina ja kuvina.
- **Ulkoiset komponentit** – TextMeshPro ja grafiikkapaketit tuotuina; lähteet ja lisenssit kirjattu GDD:hen tai README:hen.
- **Suunnittelu, toteutus ja testaus kirjastolla** – GDD:n omat päätökset ja backlog, featuret 1–7 committeina, testitapaukset T01–T12.
- **Julkaisu asiakkaan ympäristöön** – Unity WebGL -build GitHub Pagesissa; asiakas on kokeillut julkaistua linkkiä.
- **Ohjelmiston dokumentointi** – README, käyttöohje, gdd.md ja projektipäiväkirja project-docs-kansiossa.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.